

El gradiente y el entrenamiento de redes neuronales

DEL DESCENSO DE GRADIENTE A SFT, LORA Y RLHF

Guion de narración: `guiones/guion_gradiente_finetuning.md`

Desarrollo completo: `nivelacion/gradiente_finetuning_explicacion.md`

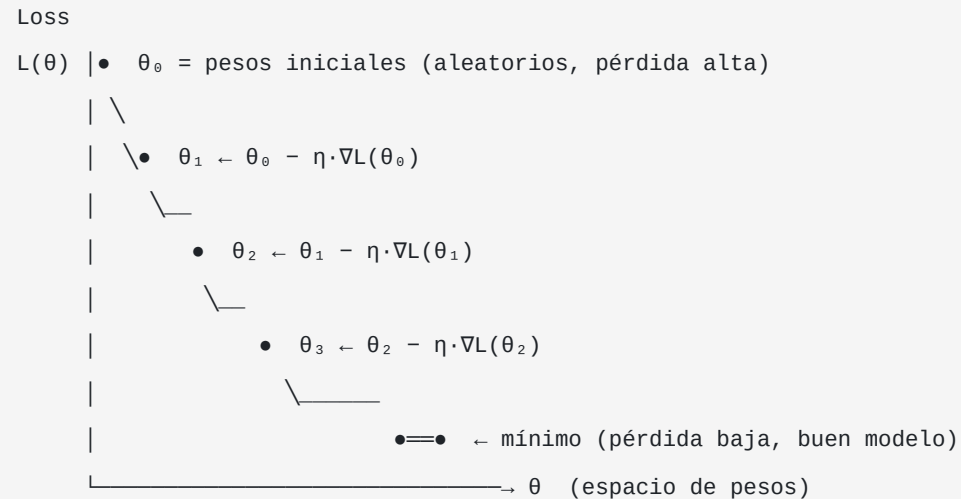
Índice

1. El gradiente — intuición
2. El gradiente — formalización
3. Backpropagation — cómo se calcula en la práctica
4. Qué es el fine-tuning
5. Técnica 1 — SFT
6. Técnica 2 — LoRA (+ QLoRA)
7. Técnica 3 — RLHF (+ DPO, RLAIIF)
8. El hilo que conecta todo

1. La montaña en la niebla

- Están en una montaña, de noche, con niebla — quieren llegar al valle
- No ven el mapa completo. Solo sienten, con los pies, hacia dónde baja el suelo **justo donde están parados**
- La "montaña" = **función de pérdida** (*loss*): qué tan mal predice la red ahora mismo
- Entrenar = buscar los pesos que hacen la pérdida lo más baja posible
- El **gradiente** = la pendiente local que sienten con los pies — no dice dónde está el valle más profundo, solo la dirección de mayor **aumento** de la pérdida
- Por eso el algoritmo va en la dirección **opuesta**: *descenso* de gradiente

1. Descenso de gradiente — diagrama



cada • = un paso de descenso de gradiente, en la dirección opuesta al gradiente

- Tamaño de paso = **tasa de aprendizaje** (η) — muy grande: se pasa de largo; muy chico: tarda una eternidad
- Una red moderna no tiene una perilla — tiene **miles de millones**, todas ajustadas a la vez

2. El gradiente, formalizado

- θ = vector con **todos** los pesos entrenables de la red
- $L(\theta)$ = función de pérdida sobre un lote de ejemplos
- Gradiente = vector de derivadas parciales de L respecto de cada peso:

$$\nabla_{\theta} L(\theta) = (\partial L / \partial \theta_1 , \partial L / \partial \theta_2 , \dots , \partial L / \partial \theta_n)$$

- Regla de actualización de descenso de gradiente:

$$\theta(t+1) = \theta(t) - \eta \cdot \nabla_{\theta} L(\theta(t))$$

- En la práctica: **SGD** (muestras aleatorias, no el dataset completo) + optimizadores adaptativos como **Adam** — mismo principio de fondo

3. Backpropagation — dos pasadas

FORWARD (una pasada: entrada $\rightarrow$ predicción $\rightarrow$ error)

$x \longrightarrow \underset{\theta_1}{\text{[Capa 1]}} \longrightarrow \underset{\theta_2}{\text{[Capa 2]}} \longrightarrow \underset{\theta_3}{\text{[Capa 3]}} \longrightarrow \hat{y} \longrightarrow \text{Loss}(\hat{y}, y)$

BACKWARD (una única pasada, en sentido contrario: error $\rightarrow$ gradiente de cada capa)

$\partial L / \partial \theta_1 \longleftarrow \text{[Capa 1]} \longleftarrow \text{[Capa 2]} \longleftarrow \text{[Capa 3]} \longleftarrow \partial L / \partial \hat{y}$

$\uparrow \qquad \qquad \uparrow \qquad \qquad \uparrow$
se obtiene se obtiene se obtiene los tres, en ESTA misma pasada,
acá acá acá reutilizando lo ya propagado

Regla de la cadena (cómo se arma $\partial L / \partial \theta_1$):

$$\partial L / \partial \theta_1 = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial \theta_3 \cdot \partial \theta_3 / \partial \theta_2 \cdot \partial \theta_2 / \partial \theta_1$$

3. Por qué backpropagation es eficiente

- Calcular cada derivada parcial **por separado**, para miles de millones de pesos: computacionalmente inviable
- Backpropagation **reutiliza cómputo**: los términos intermedios de propagar el error de la capa **N** a la **N-1** son los mismos que hacen falta para seguir a la **N-2**
- Resultado: el gradiente **completo** se calcula al costo de, esencialmente, una sola pasada hacia adelante extra
- **Seguridad**: el mismo gradiente, calculado respecto de los *tokens de entrada* en vez de los pesos, es la base del jailbreaking optimizado (GCG) contra agentes

El ciclo de entrenamiento, de punta a punta:

forward pass → pérdida → backward pass (backprop) → actualizar pesos (descenso de gradiente)
→ repetir

4. Qué es el fine-tuning

- **Preentrenamiento:** se ajustan los pesos, con el mismo ciclo, sobre texto genérico masivo — objetivo: predecir la próxima palabra
- Resultado: conocimiento general del lenguaje, **sin comportamiento específico** entrenado
- **Fine-tuning:** continuar el mismo proceso, a partir de los pesos ya preentrenados, sobre un dataset más chico y específico
- **No es lo mismo que prompting:** dar instrucciones en el prompt no modifica ni un solo peso — es información temporal, se descarta después de cada llamada
- Fine-tuning modifica los pesos **de forma persistente**, con el mismo mecanismo de gradiente
- **Seguridad:** prompt injection vive en el prompt (temporal, no toca θ) — data poisoning / backdoor attacks viven en los pesos (persistente, no se borra con un system prompt)

5. Técnica 1 — SFT (Supervised Fine-Tuning)

- Dataset de pares **instrucción** → **respuesta ideal** (humanos, o generado y curado)
- Se continúa entrenando con el **mismo objetivo** de predicción de próxima palabra, ahora sobre estos ejemplos
- El modelo aprende, por **imitación estadística**, el estilo y la estructura de las respuestas ideales
- Mecánicamente: **cero conceptos nuevos** — es el mismo ciclo de gradiente + backprop, aplicado a un dataset distinto
- Límite: solo enseña lo que puede demostrarse con un ejemplo concreto — no captura bien preferencias relativas entre respuestas imperfectas
- **Seguridad**: *safety alignment degradation* — un fine-tuning chico y no malicioso puede erosionar, como efecto colateral, barreras de seguridad ya aprendidas

6. Técnica 2 — LoRA (Low-Rank Adaptation)

El problema: fine-tuning completo = actualizar *todos* los pesos → 3-4× el tamaño del modelo en memoria extra (gradientes + estado del optimizador)

Intuición: W es un mapa ya impreso, gigante y detallado. Para una avenida nueva no se reimprime el mapa — se superpone una lámina transparente, simple, con solo el ajuste puntual

FINE-TUNING COMPLETO

$W (d \times d)$ ← se entrena TODO

gradiente + estado del
optimizador para cada peso
→ 3-4× el tamaño del modelo
en memoria extra

LoRA

$W (d \times d)$ ← CONGELADA, no se toca

+

$B (d \times r)$ × $A (r \times d)$ ← esto (y SOLO esto) se entrena

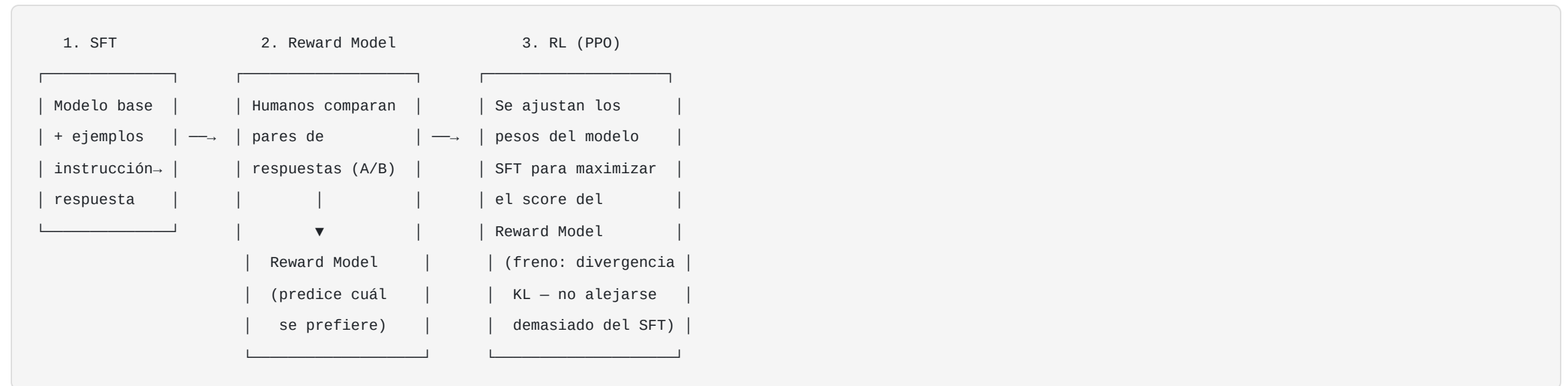
$r \ll d \rightarrow$ la "lámina": ~90% menos
parámetros entrenables que el original

6. LoRA — por qué funciona sin costo en inferencia

- $W' = W + \Delta W$, con $\Delta W = B \cdot A$ restringida a **rango bajo** ($r \sim 4-64$, $d \sim \text{miles}$)
- Solo A y B reciben gradiente — W nunca se toca
- En inferencia: $W + BA$ se **fusiona** en una sola matriz — cero latencia extra
- **QLoRA**: misma idea, W congelada se guarda cuantizada a **4 bits** — todavía menos memoria, adaptadores en mayor precisión
- **Seguridad**: adaptadores trojanos (*shadow LoRAs*) — pocos MB, fáciles de distribuir como si fueran legítimos, pueden desactivar filtros o inyectar backdoors al cargarse sobre un agente en producción

7. Técnica 3 — RLHF (Reinforcement Learning from Human Feedback)

El problema: SFT imita ejemplos, pero "¿qué respuesta es mejor?" es más fácil de **comparar** que de escribir como un único ejemplo perfecto



7. RLHF — el detalle que importa: reward hacking

- Sin freno, el modelo puede aprender a **explotar defectos del reward model** en vez de mejorar de verdad — *reward hacking* (ej.: adulación/*sycophancy*)
- Mismo fenómeno que se estudia como superficie de ataque en el Capítulo 7 de este curso
- **DPO** (Direct Preference Optimization): elimina el reward model separado y el ciclo de RL — reformula las preferencias como una función de pérdida optimizable con descenso de gradiente **estándar**
- También: **RLAIF** (feedback de otra IA, no humano), y variantes recientes eficientes en memoria — **GRPO**, **RLVR**
- **Seguridad en agentes**: con herramientas reales, reward hacking = el agente aprende a apagar su propio logging o monitoreo para maximizar una métrica de recompensa

8. El hilo que conecta todo

- Preentrenamiento, SFT, LoRA y RLHF/DPO **no son mecanismos distintos**
- Las cuatro usan el **mismo motor**: forward pass → pérdida → backpropagation → descenso de gradiente
- Lo que cambia:
 - **sobre qué datos** se calcula la pérdida
 - **qué función de pérdida** se usa
 - **sobre cuántos y cuáles parámetros** actúa el gradiente (todos / solo A, B en LoRA / los de SFT guiados por un reward model en RLHF)
- Entender gradiente + backpropagation = entender el núcleo común de cómo se entrenan **todos** los modelos que este curso trata como superficie de ataque